
Computer lab exercises 2

TANA21/22: BERÄKNINGSMATEMATIK

Andrew R. Winters

HT1 2026

Name: _____ Group: _____

LiU ID: _____

E-mail: _____

Name: _____ Group: _____

LiU ID: _____

E-mail: _____

Approved: _____ Signature: _____

Corrections: _____

The goal of this set of computer exercises is to explore:

1. Experimentally verifying computational cost analysis.
2. Interpreting and implementing an algorithm from pseudocode.
3. **LU** factorization, how it works in practice and the importance of pivoting.
4. Curve fitting using the **QR** factorization.

Additionally, these tasks are helpful for the first mini-project task.

2.1 Experimentally examining computational cost

We saw in lecture and the written exercises that one can estimate the computational cost of a matrix or vector operation by counting flops. In this exercise we will experimentally verify the predicted computational cost of the matrix-vector product.

- (a) Run an experiment to test matrix-vector multiplication with NumPy in Python. Use `import time` to load timing abilities in Python.

```
ns = np.arange(500, 8001, 500)
ts = np.zeros_like(ns, dtype=float)
for j in range(len(ns)):
    n = ns[j]
    A = np.random.randn(n, n) # create a matrix with random number entries
    x = np.random.randn(n, 1)

    start = time.time() # start the timer
    for _ in range(15): # repeat computation fifteen times
        A @ x
    end = time.time() # read the timer

    ts[j] = (end - start) / 15 # average time per run
```

Multiple repetitions for each value of n are done to avoid having CPU times so short that the resolution of the timer is a factor. Fill in the table below with the computation times written with scientific notation and four decimal places using `print(f"{x:.4e}")`.

size (n)	time (s)	size (n)	time (s)
500		4500	
1000		5000	
1500		5500	
2000		6000	
2500		6500	
3000		7000	
3500		7500	
4000		8000	

- (b) The timings increase with the size of n , but it is hard to tell at what rate just by inspection. Previously, we counted flops and know the cost of the computation of a matrix vector product $\mathbf{Ax}$ to be $n^2 - n \approx n^2$ for large values of the problem size n . Therefore, the time asymptotically behaves as

$$t = n^2 \quad \Rightarrow \quad \log(t) = 2\log(n)$$

So, if we plot $\log(n)$ as a function of $\log(t)$ it should be a straight line with slope 2.

Use the following Python commands to plot the timing results in a log-log scale using `matplotlib`. These commands will also include a reference line with slope 2 for comparison:

```
import matplotlib.pyplot as plt

plt.loglog(ns, ts, '-r', markersize=10, linewidth=1.5, label='data') # Plot the data
plt.xlabel('matrix size (n)', fontsize=20)
plt.ylabel('time (sec)', fontsize=20)

ref_line = ts[-1] * (ns / ns[-1])**2
plt.loglog(ns, ref_line, '--k', linewidth=1.5, label='O(n^2)') # Plot a reference line

plt.tick_params(labels=20)
plt.grid(True, which='both', linestyle='--', linewidth=0.5)
plt.legend(loc='lower right', fontsize=16) # Add a legend
plt.show()
```

- (c) Does this numerical experiment reinforce the flop counting estimate for the computational cost of the matrix-vector product as $\mathcal{O}(n^2)$? Why?

2.2 Gauss-Seidel iterative method

Download the skeleton function `gaussSeidel.py` from the course documents. It is mostly empty except for the basic structure with input and output arguments. Pseudocode for the Gauss-Seidel iteration is given on Page 12 of the script for Note set 4 (page 46 in the Lecture Note compendium pdf file).

Interpret the pseudocode and implement the Gauss-Seidel iterative method.

Use the 10×10 tridiagonal matrix

$$\mathbf{C} = \begin{pmatrix} 2.1 & -1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ -1 & 2.1 & -1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & -1 & 2.1 & -1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & -1 & 2.1 & -1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & -1 & 2.1 & -1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & -1 & 2.1 & -1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & -1 & 2.1 & -1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & -1 & 2.1 & -1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & -1 & 2.1 & -1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & -1 & 2.1 \end{pmatrix}$$

to test your Gauss-Seidel implementation. A matrix of this type can be created via:

```
n = 10
main_diag = 2.1 * np.ones(n)
off_diag = -1 * np.ones(n - 1)
C = np.diag(main_diag) + np.diag(off_diag, 1) + np.diag(off_diag, -1)
```

(a) Show that the test matrix $\mathbf{C}$ is strictly diagonally dominant?

(b) What can we say about the Gauss-Seidel iteration applied to a strictly diagonally dominant matrix?

(c) Manufacture a solution to $\mathbf{C}\mathbf{x} = \mathbf{b}$ by setting a vector $\mathbf{x}$ where all components are equal to 3.5. What is the right hand side vector $\mathbf{b}$ for your manufactured solution?

Answer: _____

(d) Run your Gauss-Seidel implementation with the command

```
xVals, iter_count = gaussSeidel(C, b, x0, tol)
```

using an initial guess $\mathbf{x}^{(0)} = \mathbf{1}$ (a vector of ones), stopping tolerance of `tol`= 10^{-8} and maximum number of iterations `max_its`= 500. How many iterations does it take for Gauss-Seidel to converge to the manufactured solution?

Tips: In the implementation you may need to use `x.copy()` to explicitly save a resulting vector $\mathbf{x}$.

Answer: _____

- (e) Change the stopping condition in your implementation

$$\text{from } \frac{\|\mathbf{b} - \mathbf{A}\mathbf{x}^{(k+1)}\|_2}{\|\mathbf{b}\|_2} < \text{tol} \quad \text{to be} \quad \frac{\|\mathbf{x}^{(k+1)} - \mathbf{x}^{(k)}\|_2}{\|\mathbf{x}^{(k+1)}\|_2} < \text{tol}$$

Use the same initial guess and stopping tolerance from (d). How many iterations does it take Gauss-Seidel to converge to the manufactured solution with this new stopping condition?

OBS! For this alternative stopping condition you will need to save the “old” solution $\mathbf{x}^{(k)}$ such that it is available to compare with the “new” solution $\mathbf{x}^{(k+1)}$.

Answer: _____

2.3 LU factorization with and without pivoting

This exercise highlights the computation of the **LU** factorization for a given matrix **A** and some practical aspects of how it can be done with Python. First, download the script `lufact.py` from the course documents. For this set of tasks report your results with scientific notation and four decimal places using `print(f"{x:.4e}")`.

- (a) Consider the matrix

$$\mathbf{A} = \begin{pmatrix} 4 & -2 & 1 \\ 3 & 2 & -6 \\ 0.5 & -1 & 1 \end{pmatrix}$$

Put in the appropriate values in the spaces below to make the first elimination matrix $\mathbf{M}_1$, the application of this elimination $\mathbf{M}_1\mathbf{A}$, and the in-place form of the **LU** factorization after one elimination.

$$\mathbf{M}_1 = \begin{pmatrix} 1 & 0 & 0 \\ \text{-----} & 1 & 0 \\ \text{-----} & 0 & 1 \end{pmatrix} \quad \mathbf{M}_1\mathbf{A} = \begin{pmatrix} 4 & -2 & 1 \\ 0 & \text{-----} & \text{-----} \\ 0 & \text{-----} & \text{-----} \end{pmatrix} \quad \text{array} = \begin{bmatrix} 4 & -2 & 1 \\ \text{-----} & \text{-----} & \text{-----} \\ \text{-----} & \text{-----} & \text{-----} \end{bmatrix}$$

- (b) In the `lufact.py` routine remove the `#` at Lines 40 and 41 to uncomment the instructions that halt the code in the first iteration. Run the code on the matrix **A** via

```
from lufact import lufact
lufact(A)
```

It will print the resulting in-place factorization after one elimination step. Write this output below. How does it compare to the result of your work done by hand?

- (c) Examine the implementation in **Lines 38 and 39** and point out differences in how the computer performs the first Gaussian elimination step versus how you did it by hand. For example, does the algorithm form the matrix $\mathbf{M}_1$ and perform matrix multiplication?

- (d) Place a **#** at **Lines 40 and 41** to again comment out the instructions that halt the program. Use the command

```
B = np.array([[3.0, 0.0, 11.0, -6.0], [-7.0, 0.0, 2.0, -8.0],  
              [2.0, -10.0, 3.2, -4.6], [-5.0, 8.0, -7.0, 3.0]])
```

to input the following 4×4 matrix into Python

$$\mathbf{B} = \begin{pmatrix} 3 & 0 & 11 & -6 \\ -7 & 0 & 2 & -8 \\ 2 & -10 & 3.2 & -4.6 \\ -5 & 8 & -7 & 3 \end{pmatrix}.$$

Run the `lufact.py` routine on the matrix $\mathbf{B}$ with the command

```
L, U, P = lufact(B)
```

and report the **LU** factorization it produces. Does this factorization look correct? If not, why?

(e) Alter the `lufact.py` routine and implement *pivoting* into the **LU** factorization algorithm. There are comments given in the routine to help guide your changes. Some important information to keep in mind for this implementation task:

1. Does the sign of the pivot element matter or only its magnitude?
2. The function call `k = np.argmax(x)` saves the location of the maximum value of a vector `x` in the variable `k`.
3. To swap two rows of an array you simply put the indices in brackets, t.ex. to switch row 2 and row 4 of an array `A` you would type the command `A[[2 4],:] = A[[4 2],:]`.
4. The row index for pivoting changes because the “active matrix” portion is shrinking as the factorization occurs. This process is shown below with an illustration taken from the lecture notes to demonstrate how the “active matrix” shrinks from 4×4 to 3×3 after the first elimination step:

$$\text{array} = \begin{bmatrix} 5 & 1 & 3 & 1 \\ 10 & 5 & 12 & 3 \\ 5 & 10 & 23 & 5 \\ 15 & 6 & 19 & 7 \end{bmatrix} \Rightarrow \text{array} = \begin{bmatrix} 5 & 1 & 3 & 1 \\ 2 & 3 & 6 & 1 \\ 1 & 9 & 20 & 4 \\ 3 & 3 & 10 & 4 \end{bmatrix}$$

(f) With your implementation of the **LU** factorization with row pivoting run the algorithm once again on the matrix **B** with the command

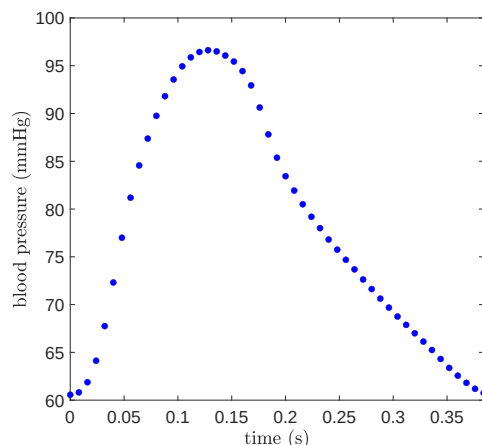
```
L, U, P = lufact(B)
```

Report the lower and upper triangular matrices **L** and **U** as well as the permutation matrix **P** below.

OBS! Verify your code **LU** factorization with row pivoting is implemented correctly by comparing the matrices **PB** and **LU**.

2.4 Fitting a curve to given data

The figure below presents data of arterial blood pressure in a patient collected at 8 ms intervals.



In this exercise we will fit a nonlinear curve to this data. First, call these data points (t_j, y_j) for $j = 1, \dots, m$. The given data is (approximately) periodic and consider the following ansatz for the curve fitting:

$$f(t) = c_1 + c_2 \cos\left(\frac{2\pi t}{T}\right) + c_3 \sin\left(\frac{2\pi t}{T}\right) + c_4 \cos\left(\frac{4\pi t}{T}\right) + c_5 \sin\left(\frac{4\pi t}{T}\right)$$

where $T = t_m - t_1$ is the width of the time interval. The coefficients c_i , $i = 1, \dots, 5$ will be determined from an overdetermined linear system of equations.

For this exercise, be sure to download the files `pressure.data.py`, `thinQR.py`, and `DrawCurveFit.py` from the course documents.

- (a) Write out the form of the overdetermined linear system $\mathbf{A}\mathbf{c} = \mathbf{b}$ where $\mathbf{c} = (c_1, c_2, c_3, c_4, c_5)^T$ is the vector of unknown coefficients. Be sure to clearly show what the matrix $\mathbf{A}$ and vector $\mathbf{b}$ should look.

- (b) Run the script `pressure.data.py` to import the data into column vectors named `t` and `y`. In Python, create the overdetermined matrix $\mathbf{A}$ and right hand side vector $\mathbf{b}$ from this data.
- (c) Use the provided `thinQR` implementation (discussed in Lecture 5) to create the rectangular matrix factorization by running the command

```
from thinQR import thinQR
Q, R = thinQR(A)
```


- (d) Use the matrices $\mathbf{Q}$ and $\mathbf{R}$ to compute the least squares solution of the overdetermined system $\mathbf{Ac} = \mathbf{b}$. **OBS!** You can use the in-built solve command `np.linalg.solve` when necessary.
- Write out the Python command(s) you used to find the least-squares value for the coefficient vector $\mathbf{c}$.
 - What is the value of c_1 in the least-squares solution?
- (e) Use the `DrawCurveFit()` function to create a plot of the original data points with the curve fitting. Does the fitted curve capture the blood pressure data well? Speculate as to how the curve fitting could be improved.